PROGILIC CRM

Comprehensive Engineering, Security & Launch Readiness Audit

Including Trello account-linking assessment and a systematic evidence review

Audit date: 5 November 2026
Scope: Uploaded project archive (260 archive entries), source code, configuration, Firestore rules, tests and engineering reports.
Authorized Trello identity requested: progilicservice@gmail.com
Audit posture: Read-only review. No production deployment, account authorization, live Trello reads/writes or CRM data mutation performed.

DECISION: NOT READY FOR PRODUCTION LAUNCH OR SHARED MULTI-USER RELEASE.
The project contains a meaningful CRM implementation and a substantial Trello integration foundation, but critical security, authorization, synchronization, account-linking and deployment gaps remain. Exposed credentials in the archive must be rotated immediately.

Prepared from static source inspection, reproducible local checks and a PRISMA-informed rapid systematic evidence review. This report does not certify live infrastructure.

1. Executive decision

The project is a substantial existing CRM with Firebase/Firestore persistence, employee workflows, AI-assisted features, Trello classification/routing, webhook handling, a review queue and synchronization mapping. Rebuilding the CRM is not indicated. The correct next step is a controlled remediation and verification phase.

Area	Assessment	Launch status
Core CRM application	Broad feature set present; source-level review only, no complete live acceptance test.	**Conditional / unverified**
Authentication & employee authorization	Firebase owner access is not equivalent to independent employee authorization; email verification is not consistently enforced.	**Blocker for shared release**
Trello integration	Webhook and read-only client foundations exist, but no OAuth flow and no production orchestration using the Trello client.	**Not complete**
Sync integrity	Atomic commit and lease-renewal methods exist in repository but are not invoked by sync service.	**Blocker**
Deployment	Node backend and static Pages hosting are not shown to be wired to a single verified callback origin.	**Unverified / blocker**
Build & tests	Syntax scan passed; dependency integrity check failed; tests/build could not execute because dependencies are not installed.	**Not certified**

Overall recommendation: do not enable the production webhook, do not run historical backfill against production, and do not share the CRM with staff under the assumption that frontend permission checks are security controls.

2. High-priority findings

2.1 Exposed Trello credentials — CRITICAL

The archive's production activation checklist contains a Trello API token and webhook signing secret in plaintext. This directly contradicts the included report's claim that the secret scan found zero secrets. Treat both as compromised because the archive has been distributed to the audit process. Revoke the exposed token, rotate the webhook secret, remove credential values from all reports/patches/history, and store replacements only in a backend secret manager. Do not put them in browser bundles, static-site build variables, source control or exported audit artifacts. Trello warns that API tokens should be treated like passwords and can grant broad account access [R3].

2.2 Production safety gate — CRITICAL

The engineering report says production activation is locked pending explicit second approval. Static inspection found no corresponding production unlock flag or explicit-approval gate. The webhook request handler can proceed into sync when valid production configuration is present. The sandbox guard is not a production lock: it returns true when NODE_ENV is not test and only applies limited checks in test/sandbox. Implement a server-side fail-closed gate on webhook ingestion, backfill and every mutation path. Keep the Sunday-card scenario in isolated test storage until a separate explicit authorization is given.

2.3 Sync transaction / fencing gap — CRITICAL

FirestoreAdminRepository implements acquireSyncLease(), renewSyncLease(), verifyLeaseOwnership(), commitProtectedSyncTransaction() and releaseSyncLease(). However, trelloSyncService.js uses lease acquisition and a separate verifyLeaseOwnership() check, then writes client records, subitems, timeline activity and mapping through independent operations. The sync service does not call commitProtectedSyncTransaction() or renewSyncLease(). This creates a time-of-check/time-of-use race and permits partial writes if one step fails. Route the full business mutation and mapping update through one Firestore transaction that validates lease token, generation and expiry inside the transaction. Renew long-running leases and release them safely on all terminal paths.

2.4 Shared staff authorization — CRITICAL for sharing

Employee login and permission logic are implemented primarily in browser-side employeeService/store code. Firestore rules authorize the owning Firebase UID; employee records are not independently bound to Firebase identities or enforced custom claims. A staff user operating through the owner's authenticated session can potentially bypass UI checks and access data directly through Firestore. Implement independent staff authentication and server-enforced authorization (for example, Firebase Auth identities/custom claims with tested tenant/role rules or a trusted backend API). Never rely on

hidden UI controls as access control.

2.5 Firestore rules — HIGH

The isVerified() helper is not used by the allow statements and contains an unconditional isSignedIn() alternative, making it ineffective as an email-verification requirement. Actual user and client/subcollection rules mostly rely on isOwner(). A recursive client-subcollection rule permits owner read/write across arbitrary descendant subcollections without applying the collection-specific validators. Align the deployed rules with the written security specification, remove the permissive alternative, require verified identity where intended, and apply schema allowlists to every writable path. Firebase documents email_verified as a token claim suitable for rule checks [R9].

2.6 Trello account linking — CRITICAL

No Trello OAuth authorization-start route, OAuth callback route, authorization-code exchange, PKCE validation, state/CSRF validation, refresh-token lifecycle, or encrypted token persistence was found in the archive. server.js exposes the webhook endpoint and AI routes, not an OAuth callback. A webhook endpoint is not an OAuth callback. Therefore progilicservice@gmail.com is not proven linked to Trello by this codebase. The backend should use Trello OAuth 2.0 confidential-client flow with PKCE and offline_access where appropriate; store tokens encrypted server-side; validate state, redirect URI and identity; bind the grant to the configured Firebase UID and allowed CRM email; and reject every other CRM principal. Atlassian identifies confidential clients as the right option for apps with a backend [R4, R5].

2.7 Deployment callback mismatch — CRITICAL

The project includes a Node HTTP server (server.js) that handles /api/trello/webhook. The deployed CRM URL previously discussed is a Cloudflare Pages site, while the engineering report targets Cloud Run for the backend. No Pages Functions route or equivalent serverless adapter is included in the archive. A static Pages deployment will not automatically run this Node server. Use a deployed backend URL that serves the exact webhook and OAuth callback paths, supports HTTPS and responds to Trello's initial HEAD verification with 200. The earlier attempted API callback path returned 'Cannot GET' in the previous troubleshooting context; it is not proof of a working OAuth callback.

2.8 Trello sync is not wired end-to-end — HIGH

TrelloClient is only instantiated in its unit tests; no production import/use was found. The client can read boards/lists/cards/checklists, but the webhook handler processes incoming events and the server has no route that starts a controlled initial read/backfill using TrelloClient. The modal's dry-run uses hardcoded synthetic checklist items. The backfill UI passes arguments inconsistent with executeApprovedBackfill(tenantId, runId, items, approvedBy), so the actual UI execution path is not correctly wired. Correct the call contract, source dry-run input from real authorized read-only Trello data only after authorization, and keep production execution disabled pending approval.

2.9 Sandbox network isolation — HIGH

safeTrelloFetch() checks the host and blocks non-GET requests only when in test/sandbox mode. Its purported network-isolation branch contains no blocking action; fetch() still executes. Therefore the tests do not prove hermetic network isolation. Enforce an injected transport/mock or hard-deny outbound network unless an explicit isolated test adapter is used. Also require HTTPS, validate allowed path/resource IDs and redact credential-bearing query strings from logs.

2.10 Other correctness and operational gaps — HIGH/MEDIUM

The Trello principal check rejects email_verified only when it is explicitly false; a missing claim is accepted. Require true. The source's in-memory fallback sync/review stores are process-local and not durable; production must fail closed if durable Firestore is unavailable. TrelloClient does not implement retry/backoff for 429/5xx, bounded pagination, or operational alerting. Trello documents per-key/token rate limits and recommends webhooks to reduce polling [R8]. The app's AI classifier and sync path also need schema validation of model output before persistence, confidence thresholds, and auditability of manual overrides.

3. Verification performed

Check	Observed result	Interpretation
Archive inventory	260 entries; source, rules, tests, reports and Trello files present.	Sufficient for static source audit; not live deployment proof.

Check	Observed result	Interpretation
JavaScript syntax scan	node --check across src/ and tests/: exit 0.	Syntax only; does not validate imports/runtime behavior.
Dependency integrity	Failed: @babel/parser imported by tests/unit/i18n-duplicate-keys.test.js but not declared in package.json.	Fix dependency declaration and regenerate lockfile.
Unit tests	Could not run: vitest not installed in audit environment.	No independent pass/fail certification.
Production build	Could not run: vite not installed in audit environment.	No build artifact verification.
Prior Trello report	Claims 106/106 tests passed across 12 Trello/security test files.	Historical report, not rerun here; not the full 36 test files in archive.
Live Trello/Firebase	No live account authorization, network read, webhook registration or production write attempted.	Must be tested separately with explicit approval.

The archive contains 36 test files. The 106/106 result in the embedded Trello mission report is limited to its listed 12 Trello/security test files. It must not be represented as a fresh full-project test run.

4. Trello account-linking target architecture

Required identity boundary: only the Firebase-authenticated CRM account progilicservice@gmail.com may initiate or use the Trello connection. The server must derive the tenant UID from the verified Firebase ID token and compare it to a server-side allowlist; it must never accept tenantId from a browser request or webhook body.

Step	Required behavior
1. Start authorization	Authenticated CRM owner starts Trello authorization. Server generates cryptographically random state and PKCE verifier/challenge; state is bound to a short-lived server-side session.
2. Trello consent	Redirect to Atlassian authorization using the configured confidential OAuth 2.0 client, exact registered redirect URI, minimum read scope (initially read:board:trello and any additional identity scope strictly required), and offline_access if refresh is required.
3. Callback	Backend validates state, exact redirect URI, one-time code, PKCE verifier and OAuth response. Never accept access tokens from query fragments or browser storage as trusted credentials.
4. Identity binding	Verify the Trello identity and ensure the connection is explicitly associated with the allowlisted CRM Firebase UID/email. Do not let a second CRM account reuse the connection.
5. Secure token storage	Encrypt access/refresh tokens with KMS/secret-management-backed envelope encryption; persist only on backend; restrict IAM; rotate and revoke on disconnect.
6. Scope validation	Read the configured board/list and verify IDs and workspace membership. Enforce board/list allowlists server-side on every event and API request.
7. Webhook registration	Register webhook against the exact board/list model with backend token; expose HTTPS endpoint that passes Trello HEAD validation and verifies HMAC over raw request body + exact callback URL.
8. Safe ingestion	Persist event receipt/idempotency key, enqueue work, classify untrusted Trello text, route deterministically, send ambiguous cases to review, and atomically commit CRM writes with mapping/audit data.
9. Monitoring/disconnect	Monitor token refresh failures, webhook delivery, retries, 429/5xx, dead-letter queue and sync drift. Provide owner-only disconnect/revoke and reauthorization.

Atlassian's current Trello documentation describes OAuth 2.0 three-legged authorization for user-facing apps, PKCE, confidential clients for backend apps, and refresh-token rotation. Access tokens are short-lived; refresh tokens are single-use and expire, so refresh handling must be transactional and race-safe [R4, R5, R6].

The current app registration was previously configured as Public. Because this CRM has a backend, change the Trello OAuth client configuration to Confidential before implementing the backend authorization-code flow. Public clients are intended for front-end-only apps without a backend [R4].

5. CRM launch readiness controls

Control	Minimum acceptance criteria
Authentication	Verified email required; MFA recommended for owner/admin; session revocation and password reset tested.
RBAC	Independent employee identity; server/database enforcement; negative tests for each role and each sensitive operation.
Tenant isolation	Cross-tenant read/write tests against deployed Firestore rules; no client-controlled tenant routing.
Data integrity	Schema validation; immutable IDs; idempotent writes; migrations tested on anonymized production-like copy; backup/restore rehearsal.
Sync integrity	Atomic mutation + sync mapping + audit record; lease renewal/fencing; replay, duplicate, stale-worker and partial-failure tests.
Observability	Structured logs with correlation IDs; no tokens/PII leakage; metrics, alerting, retry queue and incident runbook.
Deployment	Backend hosting confirmed; health/readiness checks; TLS; secrets manager; staging/prod separation; rollback procedure.
Sharing readiness	User onboarding, least privilege, privacy notice, export/delete policy, support process and documented operational ownership.

6. Systematic literature review (SLR)

6.1 Review question

What technical, organizational, data-governance, security and integration capabilities are consistently associated with successful CRM implementation and reliable third-party workflow integration, and how does the uploaded PROGILIC CRM compare?

6.2 Method and limitations

A PRISMA-informed rapid systematic evidence review was performed on 5 November 2026. Search concepts covered CRM implementation critical success factors, CRM adoption/readiness, data quality and integration, Trello REST API authorization, webhooks, OAuth 2.0, secure software development and application security verification. Search sources included indexed scholarly publisher records and official primary documentation from Atlassian/Trello, Firebase, NIST and OWASP. Inclusion: peer-reviewed reviews/empirical studies or primary standards/vendor documentation directly relevant to CRM readiness, integration or secure release. Exclusion: generic CRM marketing, unrelated product material and undated secondary summaries where primary sources were available.

This is a rapid evidence synthesis, not a claim of a fully exhaustive database SLR. Full Scopus, Web of Science, IEEE Xplore and ACM searches, deduplication exports, two independent screeners and formal risk-of-bias scoring were not available in this audit environment. PRISMA 2020 recommends transparent reporting of full search strategies, study selection and flow counts [R1]. Accordingly, the review reports the evidence actually retrieved and does not fabricate screening totals.

6.3 Selected evidence and synthesis

Evidence	Key result	Implication for PROGILIC
Meyliana et al. (2016), SLR of 53 studies (2001–2014) [R2]	Technology selection and adaptation/readiness were the most dominant CRM implementation success factors.	The existing Firebase/Trello architecture is a strength, but integration completeness and deployment capability must be proven, not inferred from code presence.
Rahimi & Berman (2009), 821 questionnaires + 20-company qualitative study [R10]	CRM success depends on customer-oriented strategy and organizational process alignment, not technology alone.	Define ownership, operational workflows, employee adoption, escalation and customer-data stewardship before rollout.
2018 CRM CSF review [R11]	Synthesizes CRM implementation success factors and proposes metrics to monitor them.	Use measurable acceptance criteria: data completeness, user adoption, resolution time, sync success, duplicate rate and data accuracy.
TOE-based CRM adoption study (2019) [R12]	Data quality/integration and top-management support affect CRM evaluation/adoption; technology competence affects adoption and routinization.	Prioritize clean mappings, integration reliability, owner sponsorship and training.
Tagbo et al. (2025 online), SLR of CRM adoption in project-based organizations [R13]	Organizational barriers and organizationally centered success factors recur; review initially retrieved 256 articles.	Employee permissions, process clarity, training and change management are launch requirements, not optional polish.
Atlassian Trello API docs (2026) [R3–R8]	Delegated authorization, scoped access, webhook callback validation, HMAC signatures, retries and rate limits are defined platform behaviors.	Implement proper OAuth/backend token lifecycle, verified webhook endpoint, idempotency, retries and rate-limit handling.
NIST SSDF 1.1 [R14] and OWASP ASVS 5.0 [R15]	Release readiness requires secure environments, protected software, secure production and vulnerability response; ASVS supplies testable application security requirements.	Create a release gate with repeatable security tests, provenance, dependency integrity, secrets management and response procedures.

6.4 Evidence-derived readiness dimensions

Technology: reliable hosting, secure authentication, supported dependencies, observability and tested integrations. **Data:** quality, ownership, deduplication, migration and recoverability. **Organization:** executive ownership, role clarity, employee training and adoption. **Process:** documented workflows, exception handling, escalation and review. **Security:** least privilege, verified identities, tenant isolation, secret protection and secure release practices. **Integration:** delegated authorization, scopes, idempotency, retries, webhook verification and reconciliation.

The literature consistently supports a socio-technical readiness model: CRM success is not achieved by a functioning UI or API alone. Organizational alignment, process adoption, data quality and technical integration must mature together.

7. Prioritized remediation plan

Priority	Action	Exit evidence
P0 — Immediate	Revoke exposed Trello token; rotate webhook secret; remove secrets from checklist, patches, logs and archive; inspect access logs for token use.	Old credentials invalid; new secrets in backend secret manager; clean artifact scan.
P0 — Security gate	Add server-side production activation gate covering webhook, backfill and every write path; preserve second-approval requirement.	Negative tests prove production writes remain blocked without explicit unlock.
P0 — Identity	Require email_verified === true; enforce single allowed Firebase UID/email; implement staff identity and server-side RBAC.	Unauthorized principal and role tests return 401/403 at backend and Firestore.
P0 — Firestore	Repair verification helper; remove recursive validation bypass; apply strict schemas; test deployed rules.	Emulator suite passes all allow/deny cases; cross-tenant tests fail closed.
P0 — Sync integrity	Refactor sync to atomic protected commit; use lease renewal, fencing and release; remove non-transactional partial write path.	Race/replay/stale worker/failure injection tests prove one consistent result.
P1 — OAuth	Use Confidential Trello OAuth 2.0 client; implement PKCE, state, callback, code exchange, refresh and encrypted token persistence.	Only progilicservice@gmail.com can link; callback and refresh tests pass.
P1 — Hosting	Deploy backend on Cloud Run or another Node-capable backend; point callback URL to that backend; keep Pages frontend separate if desired.	Live HEAD returns 200; HTTPS; readiness and callback smoke tests pass.
P1 — Workflow	Wire TrelloClient to a controlled ingestion/backfill service; fix modal backfill arguments; separate synthetic tests from live reads.	Read-only board/list verification; dry-run report; no writes before approval.
P1 — QA	Declare @babel/parser; install from lockfile; run integrity, lint, all unit/integration/E2E/performance tests and production build.	Clean reproducible CI run with artifacts and test counts.
P2 — Operations	Add rate-limit backoff, webhook retry/dead-letter handling, monitoring, backups, restore drills and operator runbooks.	Operational readiness checklist signed off; recovery exercise recorded.

8. Release decision and acceptance gate

Current decision: NO-GO. The CRM should not be declared production-ready for shared multi-user use, and the Trello integration should not be activated. The project can proceed to a remediation/staging phase without redesigning the existing CRM.

A GO decision requires all P0 actions closed, all tests rerun from a clean install, a production build produced, Firestore rules tested in emulator and deployed, OAuth account identity verified, backend callback live and HEAD-valid, a read-only Trello board/list verification completed, and a separate explicit approval before any production CRM mutation or live synchronization.

No real Trello account was linked during this audit. The uploaded archive does not contain a complete OAuth flow, and the audit intentionally did not use the exposed credentials or perform any live authorization or synchronization.

9. References

[R1] Page MJ et al. (2021). The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. BMJ 372:n71. https://www.bmj.com/content/372/bmj.n71

[R2] Meyliana, Hidayanto AN, Budiardjo EK (2016). The critical success factors for customer relationship management implementation: a systematic literature review. International Journal of Business Information Systems 23(2), 131–174. https://doi.org/10.1504/IJBIS.2016.078904

[R3] Atlassian Support. Get started with Trello's REST API. https://support.atlassian.com/trello/docs/getting-started-with-trello-rest-api/

[R4] Atlassian Developer. OAuth 2.0 Client Configuration (last updated 1 Oct 2026). https://developer.atlassian.com/cloud/trello/guides/rest-api/oauth-2-client-configuration/

[R5] Atlassian Developer. OAuth 2.0 Confidential Client Usage. https://developer.atlassian.com/cloud/trello/guides/rest-api/oauth-2-confidential-client-usage/

[R6] Atlassian Developer. OAuth 2.0 for Existing Apps. https://developer.atlassian.com/cloud/trello/guides/rest-api/oauth-2-for-existing-apps/

[R7] Atlassian Developer. Trello Webhooks. https://developer.atlassian.com/cloud/trello/guides/rest-api/webhooks/

[R8] Atlassian Developer. Trello Rate Limits. https://developer.atlassian.com/cloud/trello/guides/rest-api/rate-limits/

[R9] Firebase. Security Rules and Firebase Authentication. https://firebase.google.com/docs/rules/rules-and-auth

[R10] Rahimi ID, Berman U (2009). Building a CSF framework for CRM implementation. Journal of Database Marketing & Customer Strategy Management 16, 253–265. https://doi.org/10.1057/DBM.2009.29

[R11] A systematic review for the determination and classification of the CRM critical success factors supporting with their metrics (2018). Future Computing and Informatics Journal 3(2), 398–416. https://doi.org/10.1016/j.fcij.2018.11.003

[R12] Understanding CRM adoption stages: empirical analysis building on the TOE framework (2019). Computers in Industry 109, 1–13. https://doi.org/10.1016/j.compind.2019.03.007

[R13] Tagbo SK, Adekoya AF, Mensah PK (2025 online). A systematic literature review on the adoption of CRM in project-based organisations. International Journal of Society Systems Science 15(2), 170–208. https://doi.org/10.1504/IJSSS.2024.144736

[R14] NIST (2022). SP 800-218, Secure Software Development Framework (SSDF) Version 1.1. https://csrc.nist.gov/projects/ssdf

[R15] OWASP. Application Security Verification Standard 5.0.0 (30 May 2025). https://asvs.owasp.org/